

Lecture 3: Discrete Random Variables, Expectation

Jamie Haddock

Table of contents

0.1	Why counting?	1
0.2	The counting principle	1
0.3	k -permutations, combinations, partitions	2
0.4	Counting summary	2
0.5	Random variables	2
0.6	Functions of random variables	3
0.7	Discrete random variables	3
0.8	Is the randomized Kaczmarz error discrete?	3
0.9	The probability mass function (PMF)	4
0.10	Example: the PMF of Y (the dice-game payout)	4
0.11	Five named discrete distributions	4
0.12	Five named discrete distributions, continued	4
0.13	Five named discrete distributions, continued	5
0.14	Aside: Poisson as a limit of Binomial	6
0.15	Expectation	7
0.16	Compute $E[X]$ for each named distribution	7
0.17	Example: the Quiz Problem	8
0.18	Working it out directly	9
0.19	The general case	9
0.20	Interpretation: expectation as center of gravity	9

0.1 Why counting?

Many of the probability laws we'll compute with live on **finite** sample spaces where every outcome is equally likely (the *discrete uniform* law):

$$P(A) = \frac{|A|}{|\Omega|}.$$

So computing probabilities often reduces to **counting** outcomes carefully. We'll build four counting tools today: the counting principle, k -permutations, combinations, and partitions.

0.2 The counting principle

i Definition: Counting principle

Suppose a task consists of r stages performed in sequence. If stage 1 has n_1 possible outcomes, and for every outcome of stage 1 there are n_2 possible outcomes of stage 2, and so on, then the total number of possible outcomes of the r -stage task is

$$n_1 \cdot n_2 \cdots n_r.$$

Example: how many subsets does an n -element set have?

Think of building a subset $S \subseteq \{1, \dots, n\}$ as an n -stage task: for each element, decide “in S ” or “not in S ” — 2 choices, made n times in a row.

By the counting principle, the number of subsets of an n -element set is 2^n .

0.3 k -permutations, combinations, partitions

Five students in our probability reading group: **Alice, Bob, Carol, Dave, Eve**.

k -permutations. In how many ways can we choose an *ordered* 1st/2nd/3rd place finish among the 5, in a problem-solving contest?

$$5 \cdot 4 \cdot 3 = \frac{5!}{(5-3)!} = 60.$$

In general, the number of k -permutations of an n -element set is $\frac{n!}{(n-k)!}$.

Combinations. In how many ways can we choose an *unordered* team of 3 to present at a conference? Each unordered team of 3 corresponds to $3! = 6$ orders in the 5! permutations, so

$$\binom{5}{3} = \frac{60}{3!} = 10, \quad \text{in general } \binom{n}{k} = \frac{n!}{k!(n-k)!}.$$

Partitions. In how many ways can we split the five students and professor (six people in total) into one two-person team to go to conference session A, one two-person team to go to conference session B, and one two person-team to go to conference session C? Choose an ordering of the six people and divide the group into three groups of two, then account for each groups' 2! overcounting, so

$$\frac{6!}{2!2!2!} = 90, \quad \text{in general (into groups of sizes } n_1, \dots, n_r): \frac{n!}{n_1!n_2!\dots n_r!}.$$

0.4 Counting summary

Four counting formulas

Quantity	Formula
# subsets of an n -element set	2^n
# k -permutations of n elements	$\frac{n!}{(n-k)!}$
# k -combinations of n elements	$\binom{n}{k} = \frac{n!}{k!(n-k)!}$
# partitions into groups of sizes $n_1, \dots, n_r$	$\frac{n!}{n_1!n_2!\dots n_r!}$

0.5 Random variables

Definition: Random variable

Given a probability space (Ω, P) , a **random variable (RV)** is a function

$$X : \Omega \rightarrow \mathbb{R}$$

that assigns a real number $X(\omega)$ to every outcome $\omega \in \Omega$.

Notation

A random variable is a *deterministic function* — the randomness comes entirely from which ω occurs, not from X itself. We use **capital** X for the random variable (the function), and **lowercase** x for a specific numerical value X might take.

0.6 Functions of random variables

If X is a random variable and $g : \mathbb{R} \rightarrow \mathbb{R}$ is any function, then $Y = g(X)$ is also a random variable, defined by $Y(\omega) = g(X(\omega))$.

Example: Roll a fair die, $\Omega = \{1, \dots, 6\}$, and let $X(\omega) = \omega$ be the face shown. Suppose the payout rule is: win \$2 if the roll is even, lose \$1 if the roll is odd. Then $Y = g(X)$ where

$$g(x) = \begin{cases} 2 & x \text{ even} \\ -1 & x \text{ odd} \end{cases}$$

is itself a random variable on the same sample space.

0.7 Discrete random variables

Definition: Discrete random variable

A random variable X is **discrete** if its range — the set of values it can take — is finite or countably infinite.

Examples: a die roll; the payout Y above; the number of heads in n coin flips; the number of flips until the first head (range $\{1, 2, 3, \dots\}$ — infinite, but still *countable*, hence discrete).

A **continuous** random variable, by contrast, takes values in an uncountable range (an interval of $\mathbb{R}$) — we'll get there in a few lectures.

0.8 Is the randomized Kaczmarz error discrete?

Recall randomized Kaczmarz (RK), solving $Ax = b$: at each step we sample a row index $I \in \{1, \dots, m\}$ at random and update

$$x_{k+1} = x_k + \frac{b_I - a_I^\top x_k}{\|a_I\|^2} a_I.$$

I is clearly a **discrete** RV — only m possible values.

But consider $E = \|x_{k+1} - x^*\|$, the error *after* this step. For fixed x_k , E looks continuous: it's a norm of a real vector, and could in principle be any real number.

Functions of discrete RVs are discrete!

E is a **function of the discrete RV** I : $E = h(I)$ for some function h . Since I takes only m values, E *also* has a range of at most m values — even though each of those m values is an arbitrary real number, not an integer. **Discreteness is about the size of the range, not about the values themselves.**

0.9 The probability mass function (PMF)

i Definition: PMF

For a discrete random variable X , the **probability mass function** is

$$p_X(x) = P(X = x) = P(\{\omega \in \Omega : X(\omega) = x\}), \quad \text{for each possible value } x.$$

Notice the notation: p_X is indexed by the *random variable* X , and evaluated at a *candidate value* x .

Properties: $p_X(x) \geq 0$ for all x , and $\sum_x p_X(x) = 1$, where the sum is over the range of X .

To calculate $p_X(x)$: collect every outcome ω with $X(\omega) = x$, and sum their probabilities under P .

0.10 Example: the PMF of Y (the dice-game payout)

Recall X is the die roll (uniform on $\{1, \dots, 6\}$) and $Y = g(X)$ pays \$2 on even, -\$1 on odd. Grouping outcomes by their value of Y :

$$p_Y(2) = P(X \in \{2, 4, 6\}) = \frac{3}{6} = \frac{1}{2}, \quad p_Y(-1) = P(X \in \{1, 3, 5\}) = \frac{3}{6} = \frac{1}{2}.$$

0.11 Five named discrete distributions

i Bernoulli(p)

$X \in \{0, 1\}$, $p_X(1) = p$, $p_X(0) = 1 - p$, for $p \in [0, 1]$.

Examples: coin tossing, rain/no rain

i Rademacher

$X \in \{-1, +1\}$, $p_X(-1) = p_X(1) = \frac{1}{2}$.

Examples: coin tossing, rain/no rain, these appear a lot in randomized algorithms (when we often want a binary r.v. which is centered on 0)

0.12 Five named discrete distributions, continued

i Binomial(n, p)

X is the number of successes in n independent Bernoulli(p) trials. Range $\{0, 1, \dots, n\}$. Choosing *which* k of the n trials succeed is a combinations count from Part 1:

$$p_X(k) = \binom{n}{k} p^k (1-p)^{n-k}.$$

Examples: number of defective products out of n total, number of patients experiencing side effect out of n total

i Geometric(p)

X is the number of independent Bernoulli(p) trials up to *and including* the first success. Range

$\{1, 2, 3, \dots\}$:

$$p_X(k) = (1 - p)^{k-1}p.$$

Examples: number of cold calls until first sale, number of free throws until first made

0.13 Five named discrete distributions, continued

i Poisson(λ)

X counts the number of (rare) events occurring in a fixed interval of time if they occur with mean rate λ and independently of the last time they occurred. Range $\{0, 1, 2, \dots\}$:

$$p_X(k) = e^{-\lambda} \frac{\lambda^k}{k!}.$$

Examples: number of major earthquakes in a region over the past century, number of emails received over an hour

```
import numpy as np
import matplotlib.pyplot as plt
from math import comb, exp, factorial

fig, axes = plt.subplots(1, 5, figsize=(18, 3))

# Bernoulli(p)
p = 0.3
axes[0].bar([0, 1], [1 - p, p], color="steelblue")
axes[0].set_title(f"Bernoulli(p={p})")

# Rademacher
axes[1].bar([-1, 1], [0.5, 0.5], color="steelblue")
axes[1].set_title("Rademacher")

# Binomial(n,p)
n_b, p_b = 10, 0.4
ks_b = np.arange(0, n_b + 1)
pmf_b = [comb(n_b, k) * p_b**k * (1 - p_b) ** (n_b - k) for k in ks_b]
axes[2].bar(ks_b, pmf_b, color="steelblue")
axes[2].set_title(f"Binomial(n={n_b},p={p_b})")

# Geometric(p)
p_g = 0.3
ks_g = np.arange(1, 21)
pmf_g = [(1 - p_g) ** (k - 1) * p_g for k in ks_g]
axes[3].bar(ks_g, pmf_g, color="steelblue")
axes[3].set_title(f"Geometric(p={p_g})")

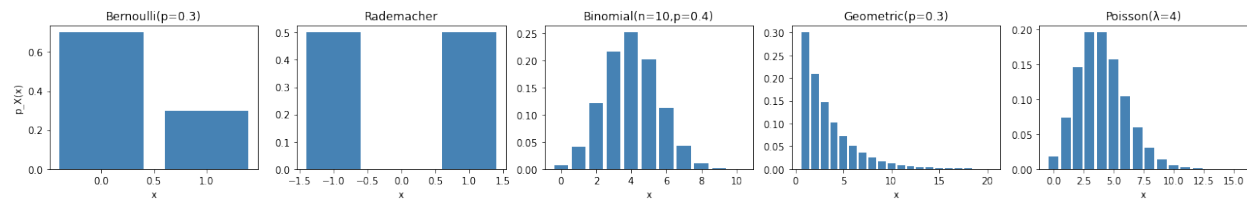
# Poisson(lambda)
lam = 4
ks_p = np.arange(0, 16)
pmf_p = [exp(-lam) * lam**k / factorial(k) for k in ks_p]
axes[4].bar(ks_p, pmf_p, color="steelblue")
axes[4].set_title(f"Poisson(={lam})")
```

```

for ax in axes:
    ax.set_xlabel("x")
axes[0].set_ylabel("p_X(x)")

plt.tight_layout()
plt.show()

```



0.14 Aside: Poisson as a limit of Binomial

💡 Fact

If $n \rightarrow \infty$ and $p \rightarrow 0$ with $np = \lambda$ held fixed, $\text{Binomial}(n, p)$ PMF converges pointwise to $\text{Poisson}(\lambda)$ PMF.

This is why Poisson is the right model for counts of rare events out of *many* opportunities for them to occur (e.g., typos on a page, radioactive decays per second).

```

import numpy as np
import matplotlib.pyplot as plt
from math import comb, exp, factorial

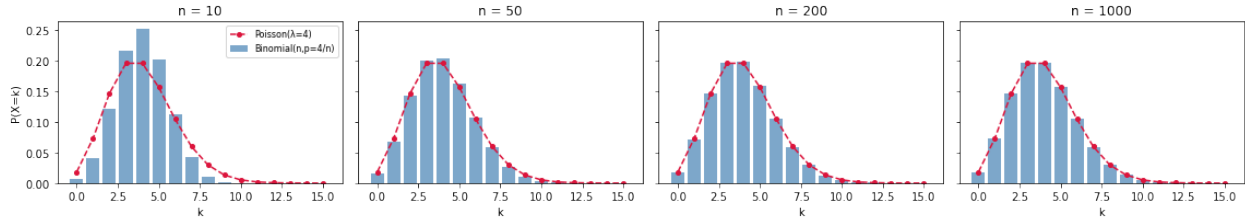
lam = 4
ks = np.arange(0, 16)
poisson_pmf = np.array([exp(-lam) * lam**k / factorial(k) for k in ks])

ns = [10, 50, 200, 1000]
fig, axes = plt.subplots(1, len(ns), figsize=(16, 3), sharey=True)

for ax, n_ in zip(axes, ns):
    p_ = lam / n_
    binom_pmf = np.array([comb(n_, k) * p_**k * (1 - p_) ** (n_ - k) for k in ks])
    ax.bar(ks, binom_pmf, color="steelblue", alpha=0.7, label=f"Binomial(n,p={lam}/n)")
    ax.plot(ks, poisson_pmf, "o--", color="crimson", markersize=4, label=f"Poisson(={lam})")
    ax.set_title(f"n = {n_}")
    ax.set_xlabel("k")

axes[0].set_ylabel("P(X=k)")
axes[0].legend(fontsize=8)
plt.tight_layout()
plt.show()

```



0.15 Expectation

Back to the dice game: win \$2 on even, lose \$1 on odd. If we played this an enormous number of times, what's the average payout per play?

Intuitively: a fraction $p_Y(2) = \frac{1}{2}$ of plays pay \$2, and a fraction $p_Y(-1) = \frac{1}{2}$ pay -\$1, so the long-run average per play should be

$$2 \cdot \frac{1}{2} + (-1) \cdot \frac{1}{2} = 0.5.$$

Definition: Expectation

The **expected value** (or **mean**) of a discrete RV X with PMF p_X is

$$E[X] = \sum_x x p_X(x),$$

summed over the range of X (assuming the sum is well defined when the range is infinite).

$E[X]$ is a single number summarizing the “center” of X 's distribution — not necessarily a value X can even take (e.g. our dice game has $E[Y] = 0.5$, but Y only ever takes the values 2 or -1!).

0.16 Compute $E[X]$ for each named distribution

Bernoulli(p): $E[X] = 0 \cdot (1 - p) + 1 \cdot p = p$

Rademacher: $E[X] = (-1) \cdot \frac{1}{2} + 1 \cdot \frac{1}{2} = 0$

In-class activity

Work with your neighbors to compute $E[X]$ for each of Binomial, Geometric, and Poisson.

Binomial(n, p):

$$\begin{aligned} E[X] &= \sum_{k=0}^n k \binom{n}{k} p^k (1-p)^{n-k} = \sum_{k=1}^n k \frac{n!}{(n-k)! k!} p^k (1-p)^{n-k} \\ &= \sum_{k=1}^n np \frac{(n-1)!}{((n-1)-(k-1))! (k-1)!} p^{k-1} (1-p)^{(n-1)-(k-1)} \\ &= np \sum_{k=0}^{n-1} \frac{(n-1)!}{((n-1)-k)! k!} p^k (1-p)^{(n-1)-k} \\ &= np \end{aligned}$$

Geometric(p):

$$E[X] = \sum_{k=1}^{\infty} k(1-p)^{k-1}p = p \sum_{k=1}^{\infty} \frac{d}{dy} y^k = p \frac{d}{dy} \sum_{k=1}^{\infty} y^k = p \frac{d}{dy} \frac{y}{1-y} = p \frac{1}{(1-y)^2} = \frac{1}{p}$$

where $y = 1 - p$

Poisson(λ):

$$E[X] = \sum_{k=0}^{\infty} k e^{-\lambda} \frac{\lambda^k}{k!} = \lambda \sum_{k=1}^{\infty} e^{-\lambda} \frac{\lambda^{k-1}}{(k-1)!} = \lambda \sum_{k=0}^{\infty} e^{-\lambda} \frac{\lambda^k}{k!} = \lambda$$

```
import numpy as np

rng = np.random.default_rng(151)
N = 500_000

p = 0.3
bernoulli = (rng.random(N) < p).astype(float)

rademacher = rng.choice([-1.0, 1.0], size=N)

n_b, p_b = 10, 0.4
binomial = rng.binomial(n_b, p_b, size=N).astype(float)

p_g = 0.3
geometric = rng.geometric(p_g, size=N).astype(float)

lam = 4
poisson = rng.poisson(lam, size=N).astype(float)

rows = [
    ("Bernoulli(p=0.3)",      bernoulli.mean(), p),
    ("Rademacher",           rademacher.mean(), 0.0),
    ("Binomial(n=10,p=0.4)",  binomial.mean(), n_b * p_b),
    ("Geometric(p=0.3)",     geometric.mean(), 1 / p_g),
    ("Poisson(=4)",          poisson.mean(), float(lam)),
]

print(f"{'distribution':<22}{'simulated E[X]':>16}{'theory':>12}")
for name, sim, theory in rows:
    print(f"{name:<22}{sim:>16.4f}{theory:>12.4f}")
```

distribution	simulated E[X]	theory
Bernoulli(p=0.3)	0.3003	0.3000
Rademacher	0.0001	0.0000
Binomial(n=10,p=0.4)	3.9963	4.0000
Geometric(p=0.3)	3.3348	3.3333
Poisson(=4)	3.9935	4.0000

0.17 Example: the Quiz Problem

A contestant is asked two questions, in an order of *their own* choosing. Get a question right and you win its prize; get it wrong and the quiz ends immediately — you forfeit everything still to come.

Say Question 1 is answered correctly with probability $p_1 = 0.8$, worth $v_1 = \$100$; Question 2 is answered correctly with probability $p_2 = 0.5$, worth $v_2 = \$200$. **Which should you answer first?**

The answer is not obvious because there is a tradeoff. You certainly want the chance to answer the more valuable Question 2, but if you attempt it first, you may never get a chance to attempt the easier Question 1. Let's look at expected winnings under the two orderings!

0.18 Working it out directly

Ask Question 1 first. You win \$100 if you get it right (probability 0.8), and *on top of that* you win \$200 if you also get Question 2 right (probability 0.8×0.5):

$$E[X] = p_1 v_1 + p_1 p_2 v_2 = (0.8)(100) + (0.8)(0.5)(200) = 80 + 80 = 160.$$

Ask Question 2 first instead:

$$E[X] = p_2 v_2 + p_2 p_1 v_1 = (0.5)(200) + (0.5)(0.8)(100) = 100 + 40 = 140.$$

0.19 The general case

Now the same question in general: Question i is answered correctly with probability p_i and is worth v_i if correct; an incorrect answer ends the quiz. **In general, which order maximizes expected total winnings?**

Exactly the same reasoning as above: if question i is asked first and j second,

$$E[X] = p_i v_i + p_i p_j v_j.$$

Asking 1 first beats asking 2 first exactly when

$$p_1 v_1 + p_1 p_2 v_2 > p_2 v_2 + p_1 p_2 v_1 \iff \frac{p_1 v_1}{1 - p_1} > \frac{p_2 v_2}{1 - p_2}.$$

Rule

Ask questions in **decreasing order of** $\frac{p_i v_i}{1 - p_i}$.

Check against our example: $\frac{p_1 v_1}{1 - p_1} = \frac{80}{0.2} = 400$ and $\frac{p_2 v_2}{1 - p_2} = \frac{100}{0.5} = 200$. Since $400 > 200$, the rule says ask Question 1 first — matching what we found directly above.

0.20 Interpretation: expectation as center of gravity

Physical picture

Place a point mass of size $p_X(x)$ at position x on the real line, for every x in the range of X . Then

$$E[X] = \sum_x x p_X(x)$$

is exactly the **center of gravity** (balance point) of this collection of masses.

```

import numpy as np
import matplotlib.pyplot as plt
from math import comb

n_b, p_b = 10, 0.4
ks = np.arange(0, n_b + 1)
pmf = np.array([comb(n_b, k) * p_b**k * (1 - p_b) ** (n_b - k) for k in ks])
EX = (ks * pmf).sum()

fig, ax = plt.subplots(figsize=(6, 3))
ax.bar(ks, pmf, color="steelblue")
ax.axvline(EX, color="crimson", linewidth=2, label=f"E[X] = {EX:.1f} (center of gravity)")
ax.set_xlabel("x")
ax.set_ylabel("p_X(x)")
ax.set_title(f"Binomial(n={n_b}, p={p_b})")
ax.legend()
plt.tight_layout()
plt.show()

```

